

Paczka — kompletny przewodnik wdrożeniowy (monorepo → multi-repo GitOps)

Dla kogo: dla osoby, która widzi projekt pierwszy raz. Ten dokument tłumaczy o co chodzi, jak to działa, co masz dziś, dokąd zmierzasz i w jakiej kolejności to zrobić — łącznie ze skryptami do napisania, ustawieniami GitHuba i pułapkami. Diagramy są w Mermaid (GitHub renderuje je natywnie), więc plik pasuje do [paczka-hub/docs/](#).

Punkt startu (dziś): JEDEN monorepo zawiera featury + Actions + docs + **materiały (dane)** razem. **Cel:** wiele repozytoriów — publiczny hub (control plane), publiczne repa featurów, **prywatne** [paczka-data](#) (materiały + sekrety).

Zasada nadrzędna, do której wszystko się sprowadza: *na lata wygrywa się nudą, nie elegancją*. Projekt utrzymują rotujący studenci, więc każda decyzja minimalizuje wiedzę plemienną, nie linijki kodu.

Spis treści

1. Model mentalny w pięciu zdaniach
2. Słownik pojęć
3. Jak to działa — warstwy i przepływ
4. Architektura docelowa — mapa repozytoriów
5. Zasady projektowe (dlaczego tak, a nie inaczej)
6. Struktura huba (drzewo katalogów)
7. Struktura template-feature (drzewo katalogów)
8. Struktura paczka-data
9. Model sekretów: age / SOPS
10. Ustawienia GitHub — co ustawić i dlaczego
11. Skrypty do napisania
12. Diagramy
13. Plan migracji: PRZED / W TRAKCIE / PO
14. Kolejność wdrażania (co najpierw i dlaczego)
15. Cykl życia: co ręcznie, co automat
16. Pułapki (footguns)
17. Lista podsumowująca (TL;DR)

1. Model mentalny w pięciu zdaniach

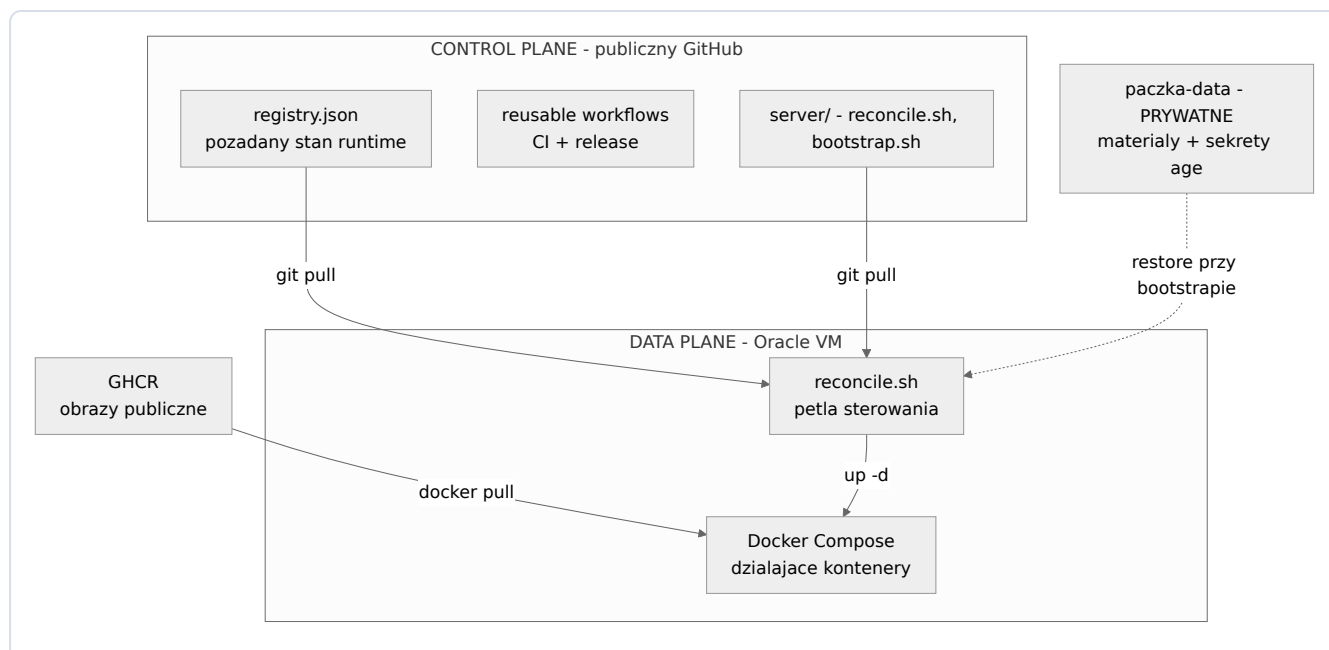
1. **registry.json** w publicznym hubie opisuje pożądaný stan wszystkich usług (którą wersję uruchomić i czy w ogóle). To jedyne źródło prawdy.
2. **Serwer sam czyta ten stan i dogania go** — pętla **reconcile.sh** na VM robi **git pull** huba i **docker compose up -d**, odtwarzając tylko to, co się zmieniło. Nikt nic nie „wypycha” na serwer.
3. **Wdrożenie to decyzja człowieka**, nie skutek merge'a: uruchamia je dopiero podbicie **version** w **registry.json**, zaakceptowane w PR przez admina.
4. **Prawie wszystko jest publiczne**, bo publiczne repo daje za darmo to, czego projekt potrzebuje: egzekwowane rulesety, nielimitowane minuty Actions, anonimowy pull obrazów z GHCR. Prywatne jest tylko **paczka-data** (materiały + sekrety).
5. **Featury nic nie wiedzą o serwerze** — to czysty kod + Dockerfile. Cała wiedza „gdzie i jak wdrażamy” żyje w hubie. Człowiek pamięta tylko dwa ruchy: *merge w featurze* i *bump w hubie*.

2. Słownik pojęć

Pojęcie	Znaczenie w tym projekcie
Control plane	Warstwa decyzji o pożądanym stanie: registry.json + workflowy + skrypty. Mieszka w hubie.
Data plane	Warstwa wykonawcza: VM w Oracle Cloud, gdzie faktycznie działają kontenery.
Reconcile	Pętla sterowania: porównuje stan pożądaný (registry) z faktycznym (docker) i koryguje różnicę.
Pull-based	Serwer sam pobiera stan i się dogania; nikt nie ma dostępu „do środka” VM.
Level-triggered	Reaguje na <i>stan</i> („jest różnica”), nie na <i>zdarzenie</i> („ktoś kliknął”). Dogania po awarii, reboocie, człowiekowi.
Idempotencja	Setny przebieg na niezmienionym registry nie robi nic.
Immutable tag	Tag obrazu vX.Y.Z nigdy się nie zmienia; :latest w produkcji zakazany.
Strangler (dusiciel)	Strategia migracji: nowy tor rośnie obok starego, przełączasz usługę po usłudze, stare archiwizujesz na końcu.
Jedna brama	Jedyny punkt, w którym coś trafia do produkcji: approve PR-a w hubie.
age / SOPS	Szyfrowanie sekretów do wielu odbiorców; wersjonowane w gicie, odszyfrowywane przy bootstrapie.

3. Jak to działa — warstwy i przepływy

Rozdziel w głowie dwie warstwy: **control plane** (gdzie zapadają decyzje) i **data plane** (gdzie się wykonują). Serwer zna wyłącznie hub i GHCR — z żadnego repa featuru nie prowadzi strzałka do VM.



Odczyt kluczowy: **stan pożądaný płynie do serwera tylko przez `git pull` huba**, a obrazy tylko przez `docker pull` z GHCR. Nic nie wchodzi do VM z zewnątrz „na żądanie”.

4. Architektura docelowa — mapa repozytoriów

Repo	Widoczność	Rola
<code>paczka-hub</code>	publiczne	control plane: <code>registry.json</code> , reusable workflows, <code>server/</code> , <code>compose/</code> , <code>docs/</code>
<code>paczka-<usługa></code>	publiczne	deployowalne featury: kod + Dockerfile + cienki caller CI/release
<code>paczka-<narzędzie></code>	publiczne	featury niedeployowalne: CI bez budowania obrazu
<code>paczka-data</code>	prywatne	materiały + zaszyfrowane (age) sekrety; zero CI
<code>paczka-monorepo</code>	publiczne	generowany , read-only zlepek (backup) z pustym <code>paczka/</code> ; produkowany przez <code>sync-monorepo</code> , nie utrzymywany ręcznie
<code>.github</code>	publiczne	profil org, szablony issue/PR
<code>template-feature-node</code> / <code>-python</code>	publiczne	template repos („Use this template”)

Uwaga o Twoim starcie: dziś wszystko (featury, actions, docs, materiały) siedzi w jednym repo. Migracja to **rozbicie** tego monorepo na powyższą mapę. `paczka-monorepo` z tabeli to *nie* Twój obecny monorepo — to nowy, generowany automatycznie backup-widok. Obecny monorepo skończy jako **prywatne archiwum** (patrz sekcja 13).

5. Zasady projektowe (dlaczego tak, a nie inaczej)

Nadrzędne - Nuda > elegancja na lata. Minimalizuj wiedzę plemienną. - **Reguła „solo”.** Repo/workflow ma sens, gdy ktoś sklonuje je *nie wiedząc, że Paczka istnieje*. To rozstrzyga granicę feature ↔ hub. - **Jedna brama wydań.** Approve PR-a w hubie — dokładnie jedna, nie dwie (stąd świadome odrzucenie GitHub Environments/Deployments API). - **Jawne > sprytne.** Wyłączenie usługi to jawny `stop`, nie poleganie na semantyce profili Compose.

Model repo i zaufania - Publiczne wszystko poza danymi. Publiczne repo daje egzekwowane rulesety, darmowe minuty Actions, anonimowy pull z GHCR, brak długożyciowych sekretów w torze CI/deploy. - **Teamy zamiast osób.** Uprawnienia wyłącznie przez `paczka-admins` / `paczka-devs` — ludzie rotują, teamy zostają. - **Feature to czysty kod.** Repo featuru nie zawiera nic o serwerze, sekretach ani wdrożeniach — to cecha, nie brak.

Load-bearing, do zapisania w README huba wprost - Merge do main huba = root na serwerze. Serwer robi `git pull` huba i wykonuje z niego `reconcile.sh` oraz pliki compose. Ruleset huba jest *de facto* kontrolą dostępu do produkcji, nie tylko higieną gitową. Dlatego zmiana `compose/` lub `server/` jest groźniejsza niż bump `registry.json`, choć przechodzi tą samą bramą — stąd osobny CODEOWNERS i deny-lista compose w CI (sekcje 10 i 11).

Control / data plane - `registry.json` = jedyne źródło prawdy: deklaratywne (opisuje stan, nie kroki), minimalne (tylko pola potrzebne reconcile), edytowane wyłącznie przez PR + approve admina. Dwie niezależne dźwignie: `version` (którą wersję) i `enabled` (czy w ogóle). - **Pull, nie push.** Brak stałego sekretu, brak ruchu przychodzącego do VM, level-triggered. - **Cron + webhook razem.** Cron/timer = kręgosłup (gwarancja zbieżności), webhook = optymalizacja opóźnienia; webhook tylko *budzi* ten sam idempotentny skrypt, nigdy nie niesie danych. - **`up -d bez down`.** Compose porównuje hash konfiguracji z etykietą kontenera → odtwarza tylko zmienioną usługę. `down` = pełny outage, zakazany.

Obrazy i wersje - Niemutowalne tagi `vX.Y.Z`, nigdy `:latest`. Niezmiennik „tag istnieje ⇒ obraz istnieje” osiągany **odwróconą kolejnością** w release: dry-run wersji → build+push → *dopiero* `git tag`. - **Tag ≠ deploy.** Auto-tagowanie jest bezpieczne, bo powstaje tylko obraz; produkcja nie rusza się, dopóki człowiek nie podbije `version`.

6. Struktura huba (drzewo katalogów)

```
paczka-hub/
├── registry.json          (PUBLICZNE – control plane)
├── registry.schema.json  # jedyne źródło prawdy o runtime
├── compose/              # walidacja registry (additionalProperties:false)
│   ├── compose.yaml     # definicje usług + profile per usługa
│   └── compose.<usługa>.yaml # (opcjonalnie) sekcje per usługa
├── server/              # to serwer pull-uje i wykonuje jako root
│   ├── reconcile.sh     # pętla sterowania
│   ├── bootstrap.sh    # gołe VM -> działający host
│   ├── lib/
│   │   ├── report.sh   # curl do webhooka Discord
│   │   └── prune.sh    # retencja obrazów + snapshotów
│   └── systemd/
│       ├── paczka-reconcile.service
│       └── paczka-reconcile.timer # OnBootSec=2min, OnUnitActiveSec=5min
├── .github/
│   └── workflows/
│       ├── hub-ci.yml   # walidacja PR-ów do huba
│       ├── ci-node.yml  # REUSABLE: CI dla featurów node
│       ├── ci-python.yml # REUSABLE: CI dla featurów python
│       ├── release.yml  # REUSABLE: dry-run -> build -> push -> tag
│       └── sync-monorepo.yml # generuje paczka-monorepo (wymaga GitHub App/PAT)
├── policy/
│   └── compose-denylist.jq # blokuje docker.sock, privileged, host-mounty
├── docs/
│   └── (ten plik + diagramy)
└── README.md             # MUSI zawierać: "merge do main = root na VM"
```

Co gdzie żyje i dlaczego: - `registry.json` + `schema` — control plane. Edytowane tylko przez PR. - `compose/` + `server/` — to jedyne rzeczy, które serwer *wykonuje*. Chronione osobnym CODEOWNERS (sekcja 10). - `.github/workflows/*` z prefiksem „REUSABLE” — wołane przez repa featurów; to jedyna zależność featur → hub, i to logiczna (caller → callee), nie runtime'owa. - `sync-monorepo.yml` — jedyne miejsce, gdzie GitHub App / techniczny PAT jest *naprawdę* potrzebny (push do cudzego repo). Reszta radzi sobie `GITHUB_TOKEN`.

7. Struktura template-feature (drzewo katalogów)

Repo z template'a to czysty kod. Test „solo”: klonujesz je nie wiedząc, że Paczka istnieje — i dalej ma sens.

```

template-feature-node/      (PUBLICZNE – "Use this template")
├─ src/                     # kod usługi
├─ Dockerfile               # jak zbudować obraz
├─ .env.example              # dokumentacja zmiennych (BEZ wartości)
├─ .github/
│   └─ workflows/
│       └─ ci.yml            # CIENKI caller: uses: paczka-hub/.github/workflows/ci-
node.yml@<sha>               # CIENKI caller: uses: paczka-
├─ release.yml              hub/.github/workflows/release.yml@<sha>
├─ .gitignore
├─ .dockerignore
└─ README.md                # co robi usługa, jak odpalić lokalnie

```

Zasady template'a: - **Zero wiedzy o serwerze/sekretach/wdrożeniu.** Żadnej ścieżki, żadnego hosta, żadnego tokena. Jeśli musisz to tu dodać — to należy do huba. - `.env.example` **dokumentuje** zmienne (nazwy, opis), nie zawiera wartości. Wartości żyją w `paczka-data`. - Callery CI/release są **cienkie** — całą logikę mają reusable workflowy huba, przypięte po SHA (nie po tagu ruchomym).

Przykładowy `ci.yml` (caller):

```

name: CI
on: [pull_request]
jobs:
  ci:
    uses: paczka/paczka-hub/.github/workflows/ci-node.yml@<PIN_SHA>
    permissions:
      contents: read

```

8. Struktura paczka-data

```

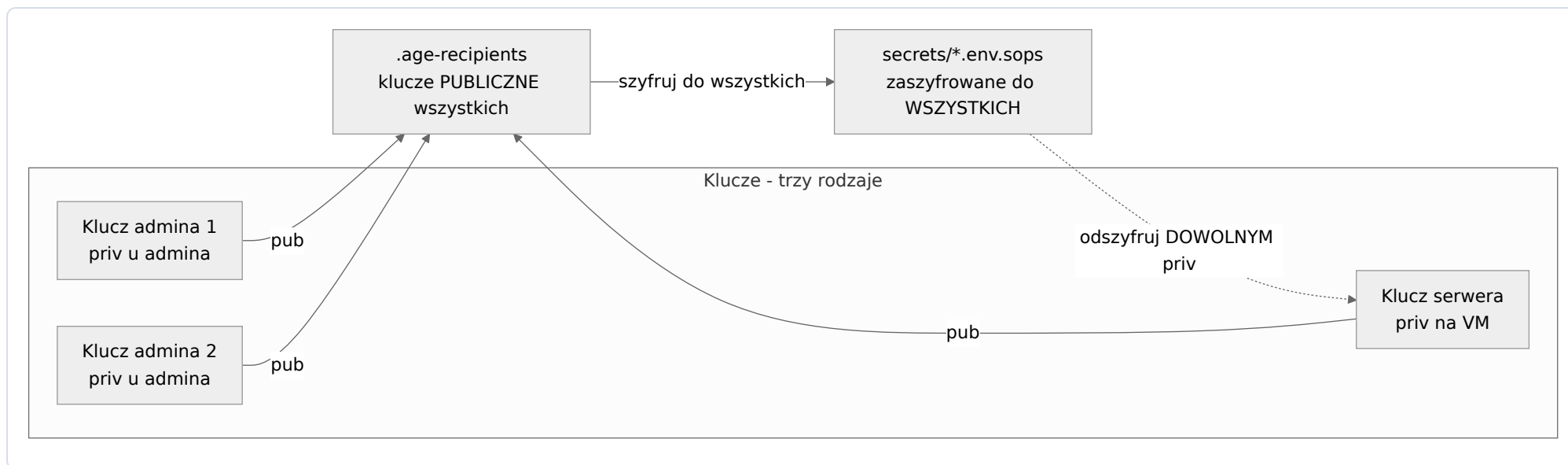
paczka-data/                (PRYWATNE – zero CI)
├─ .sops.yaml                # reguły: które pliki, którzy odbiorcy
├─ .age-recipients           # klucze PUBLICZNE adminów + serwera
├─ secrets/
│   └─ <usługa>.env.sops     # zaszyfrowane env per usługa
│       └─ ...
├─ materials/                # materiały edukacyjne (PDF, obrazy)
└─ README.md                 # jak odszyfrować, jak rotować, DR

```

Reguły: - **Zero CI** — to repo nie uczestniczy w torze CI/deploy, więc nie łamie modelu „publiczne wszystko poza danymi”. - Kierunek edycji zawsze `paczka-data` → serwer, **nigdy odwrotnie** (inaczej backup rozjedzie się ze stanem faktycznym). - Prywatność repo to **druga** warstwa ochrony sekretów, nie pierwsza — pierwszą jest szyfrowanie age.

9. Model sekretów: age / SOPS

To jest miejsce najczęstszego zamieszania — rozdzielić **trzy** rodzaje kluczy:



Jak to działa: - Każdy admin robi własną parę (`age-keygen`). **Publiczny** → `.age-recipients` (commit). **Prywatny** → u niego (menedżer haseł). Sekrety szyfrujesz do *wszystkich* recipientów naraz → dowolny admin odszyfrowuje swoim prywatnym. To jest **redundantne** — jeden admin gubi klucz, reszta dalej odszyfruje. - Serwer ma **własną** parę: publiczny w recipientach, prywatny leży na VM (wstrzyknięty przy bootstrapie). To jest „prywatny, który gdzieś musi leżeć”.

DR — i najważniejsza korekta: klucza serwera **nie musisz backupować**, bo jest **odtworzalny**. Jak VM zginie: 1. Nowy VM, admin generuje nową parę serwera. 2. Admin (swoim prywatnym) odszyfrowuje sekrety i **re-szyfruje** do `{admin1, admin2, serwer-nowy}` — jedną komendą `sops updatekeys`. 3. Bootstrap wstrzykuje nowy prywatny serwera.

Więc **jedyny nieredukowalny wymóg: przetrwa ≥ 1 klucz prywatny admina**. Realna reguła DR = „ ≥ 2 adminów, każdy trzyma swój klucz w osobnym menedżerze haseł”. Żaden pojedynczy klucz nie jest krytyczny.

Dlaczego SOPS, nie goły age: `sops updatekeys` czyni re-keying (operację, od której zależy DR i rotacja adminów) jedną komendą, plus szyfruje tylko *wartości* w YAML/JSON. To jedyne miejsce, gdzie odejście od „gołego” narzędzia zarabia na sobie.

10. Ustawienia GitHub — co ustawić i dlaczego

10.1. Organizacja i teamy

- Utwórz teamy: `paczka-admins` (approve wydań, dostęp do `paczka-data`) i `paczka-devs` (PR-y do featurów).
- Uprawnienia **tylko przez teamy** — nigdy przez konta osobiste.

10.2. Rulesety / branch protection (na każdym publicznym repo)

Na gałęzi `main` : - Wymagany PR przed merge. - Wymagany **review od CODEOWNERS**. - Wymagane **status checks** (CI musi być zielone). - **Blokada force-push** i usuwania gałęzi. - Bypass: nikt (lub tylko admini świadomie).

To działa za darmo TYLKO na repo publicznych — dlatego cały model stoi na „publiczne wszystko poza danymi”.

10.3. CODEOWNERS w hubie

```
# domyślnie: devs
* @paczka/paczka-devs

# strefy "root na VM" – wyższy rygor, tylko admini
/server/ @paczka/paczka-admins
/compose/ @paczka/paczka-admins
/registry.json @paczka/paczka-admins
/policy/ @paczka/paczka-admins
/.github/workflows/ @paczka/paczka-admins
```

10.4. Actions

- **Allowlista akcji** (Settings → Actions → Allow select actions) — żeby nikt nie dołożył losowej third-party akcji eksfiltrującej sekrety.
- **Wszystkie akcje przypięte po SHA**, nie po tagu ruchomym.
- Minimalne `permissions:` w każdym workflow (domyślnie `contents: read`).

10.5. GHCR

- Pakiety **publiczne** → anonimowy pull, bez sekretu na serwerze.
- (Opcjonalnie) scheduled workflow czyszczący stare wersje pakietów.

10.6. Webhook huba

- `push` na `main` → endpoint bota (budzi reconcile). Filtr po stronie odbiorcy: `repo == hub && ref == main`, odpowiedź `200` natychmiast, obsługa async, `flock`.

10.7. paczka-data

- **Prywatne**, dostęp tylko `paczka-admins`, **zero workflowów**.

11. Skrypty do napisania

Dla każdego: **cel**, **wejście/wyjście**, **kluczowa logika**.

11.1. `server/reconcile.sh` — pętla sterowania (serce systemu)

Cel: doprowadzić faktyczny stan dockera do stanu z `registry.json`. **Wejście:** `registry.json` (z gita) + stan dockera. **Wyjście:** odtworzone tylko zmienione usługi + raport na Discord.

Szkielet (kluczowy jest `main "$@"` w OSTATNIEJ linii — bash parsuje całość przed wykonaniem, więc `git pull` podmieniający skrypt w trakcie jest bezpieczny):

```
#!/usr/bin/env bash
set -euo pipefail

STATUS="ok"
report() { curl -fsS -X POST "$DISCORD_WEBHOOK" -d "{...status: $STATUS...}"; }
trap report EXIT # raport ZAWSZE, także przy błędzie

main() {
  exec 9>/run/paczka-reconcile.lock
  flock -n 9 || { echo "zajete, nastepny tick nadrobi"; exit 0; }

  git -C "$HUB" fetch --quiet origin main
  git -C "$HUB" reset --hard origin/main # atomowy snapshot: registry+compose+skrypt z 1
  commita

  jq -r '...' "$HUB/registry.json" > "$RUNTIME/versions.env" # wersje + aktywne profile
  # jawny STOP usług enabled:false (up -d ich NIE zatrzyma):
  docker compose --profile "$DISABLED" stop || true

  # snapshot wolumenu PRZED bumpem usługi stateful:
  for s in $CHANGED_STATEFUL; do
    docker compose stop "$s"
    tar czf "$BACKUPS/$s-$(date +%s).tar.zst" -C /var/lib/docker/volumes "$s"
  done

  docker compose --profile "$ACTIVE" pull -q
  docker compose --profile "$ACTIVE" up -d # hash configu -> tylko zmienione

  # health PER USŁUGA (nie "czy up"):
  for s in $ACTIVE; do
    h=$(docker inspect --format '{{.State.Health.Status}}' "$s")
    [ "$h" = "healthy" ] || STATUS="degraded:$s"
  done

  "$HUB/server/lib/prune.sh" # obrazy + snapshoty
}

main "$@"
```

11.2. `server/bootstrap.sh` — gołe VM → działający host

Cel: postawić hosta od zera (także w DR). **Wejście:** świeży VM + jeden sekret wstrzyknięty out-of-band (prywatny klucz age serwera). **Wyjście:** działający timer reconcile.

Kroki: instalacja `docker` + plugin `compose` + `jq` + `sops` / `age` + `flock` → wstrzyknięcie prywatnego klucza serwera → sparse-checkout huba (tylko `server/`, `compose/`, `registry.json`) → `sops -d` sekretów z `paczka-data` do `/opt/paczka/secrets/` → instalacja `systemd` timer+service → pierwszy przebieg reconcile.

Sparse-checkout to bonus dla „root na VM”: kod featurów nigdy nie ląduje na serwerze (obrazy i tak idą z GHCR), więc zawężasz, co w ogóle się wykonuje.

11.3. `.github/workflows/release.yml` (REUSABLE) — odwrócona kolejność

Cel: zbudować obraz i otagować go tak, by **tag istniał tylko, gdy obraz istnieje**. **Wejście:** merge do main featuru. **Wyjście:** obraz `:vX.Y.Z` w GHCR + tag git.

Kolejność (krytyczna): **dry-run semver** (czyta ostatni tag + Conventional Commits, *nie tworzy taga*) → **build + push** obrazu → **dopiero git tag**. `concurrency` per repo (brak wyścigów), akcje po SHA. Idempotencja: padnie build → brak taga → rerun wyliczy tę samą wersję.

To rozbraja pułapkę `GITHUB_TOKEN` (tagi pushnięte tym tokenem *nie* wyzwalają downstreamu) **bez czekania na GitHub App**.

11.4. `.github/workflows/hub-ci.yml` — walidacja PR-ów huba

Cel: żaden zły PR nie wejdzie do main huba. Sprawdza: - `check-jsonschema` — `registry.json` vs `registry.schema.json` (`additionalProperties:false` + wzorce na format wersji). - `jq` — spójność między polami (każda `enabled` usługa ma `version`, sekcję `compose` itd.). - `docker manifest inspect` — **istnienie obrazu** dla każdej zmienionej wersji (bump na nieistniejący obraz nie przejdzie). - `docker compose config -q` — składnia. - **deny-lista compose** (`policy/compose-denylist.jq` na wyrenderowanym configu) — patrz 11.6. - `shellcheck` na `server/*.sh`. - `gitleaks` — stały check, nie jednorazowy.

11.5. `ci-node.yml` / `ci-python.yml` (REUSABLE) — CI featurów

Cel: lint + test + build (bez push) na PR-ach featurów. Cienki caller w feature repo (sekcja 7) wskazuje tu po SHA.

11.6. `policy/compose-denylist.jq` — skaner złośliwych zmian

Cel: złapać eskalację uprawnień w compose, zanim admin ją przeoczy. Renderujesz `docker compose config --format json | jq -f compose-denylist.jq`; skrypt zwraca błąd, jeśli którakolwiek usługa ma: - `mount` `/var/run/docker.sock` ← najwyższy priorytet: trywialny root na hoście - `privileged: true`, `cap_add`, `security_opt` - bind-mount hosta (dozwolone tylko named volumes / allowlista ścieżek) - `network_mode: host`, `pid: host`, `ipc: host`, `usersns_mode` - odwołanie do cudzego `secrets/<inna-usługa>.env`

Granica uczciwości: to łapie eskalację uprawnień i oczywisty wyciek sekretu. NIE złapie usługi, która ma legalnie sieć i po prostu dzwoni do domu — to zostaje problemem review + CODEOWNERS, nie skanera.

11.7. `registry.schema.json`

Cel: format registry egzekwowany maszynowo. Szkic:

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "additionalProperties": false,
  "properties": {
    "services": {
      "type": "object",
      "additionalProperties": {
        "type": "object",
        "additionalProperties": false,
        "required": ["image", "version", "enabled"],
        "properties": {
          "image": { "type": "string" },
          "version": { "type": "string", "pattern": "^v[0-9]+\\.[0-9]+\\.[0-9]+$" },
          "enabled": { "type": "boolean" },
          "stateful": { "type": "boolean" },
          "profile": { "type": "string" }
        }
      }
    }
  }
}
```

11.8. `sync-monorepo.yml`

Cel: wygenerować read-only `paczka-monorepo` (backup). Jedyny workflow wymagający **GitHub App / technicznego PAT** (push do cudzego repo). Niski priorytet — na końcu migracji.

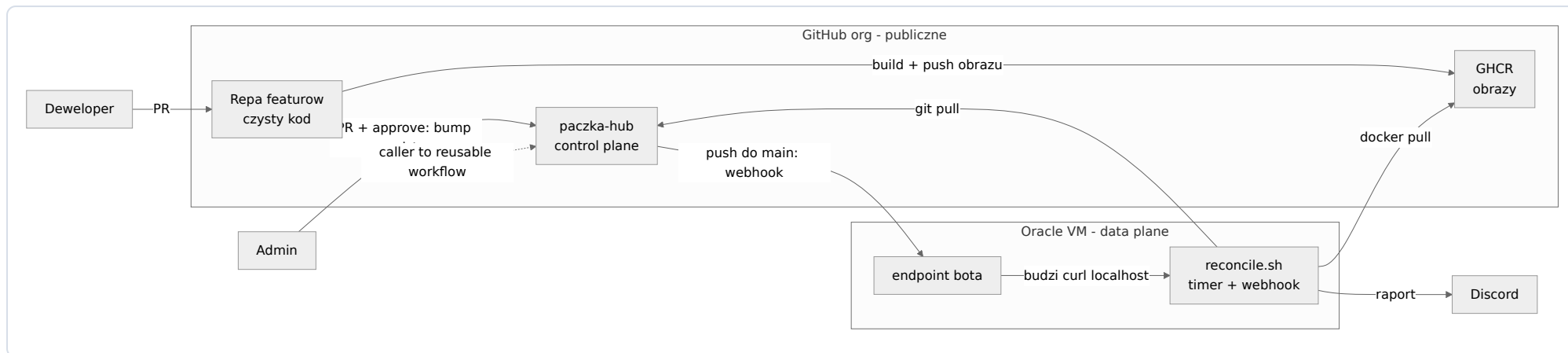
11.9. `server/lib/prune.sh` — retencja (w pętli, nie osobnym cronem)

Cel: dysk nie zapcha się za rok. Dwa źródła, nie jedno: - Obrazy: `docker image prune -af --filter "until=168h"` (bieżące immutable tagi są in-use → bezpieczne). - Snapshoty wolumenów: `find "$BACKUPS" -name '*.tar.zst' -mtime +30 -delete` (albo keep-last-N per usługa).

12. Diagramy

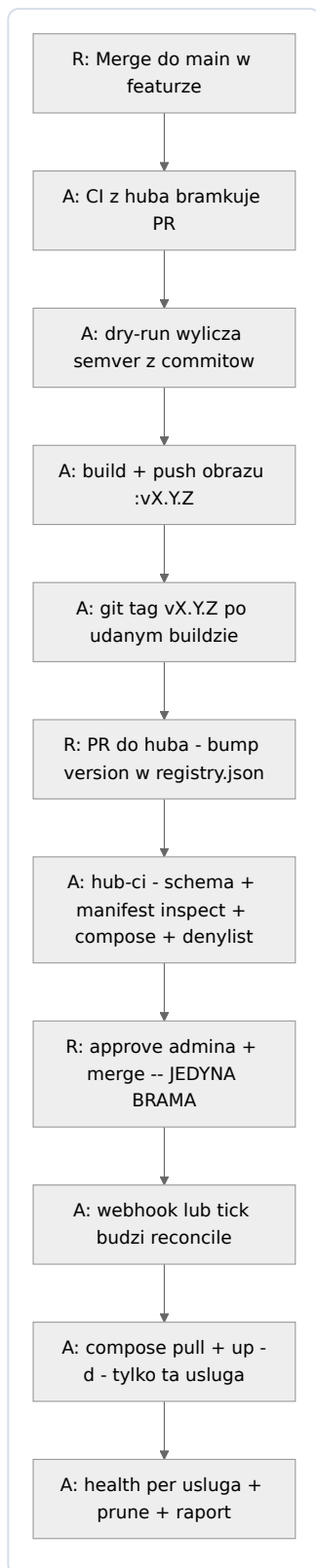
12.1. Graf zaufania w runtime

Serwer zna tylko hub i GHCR; z żadnego featuru nie ma strzałki do VM.

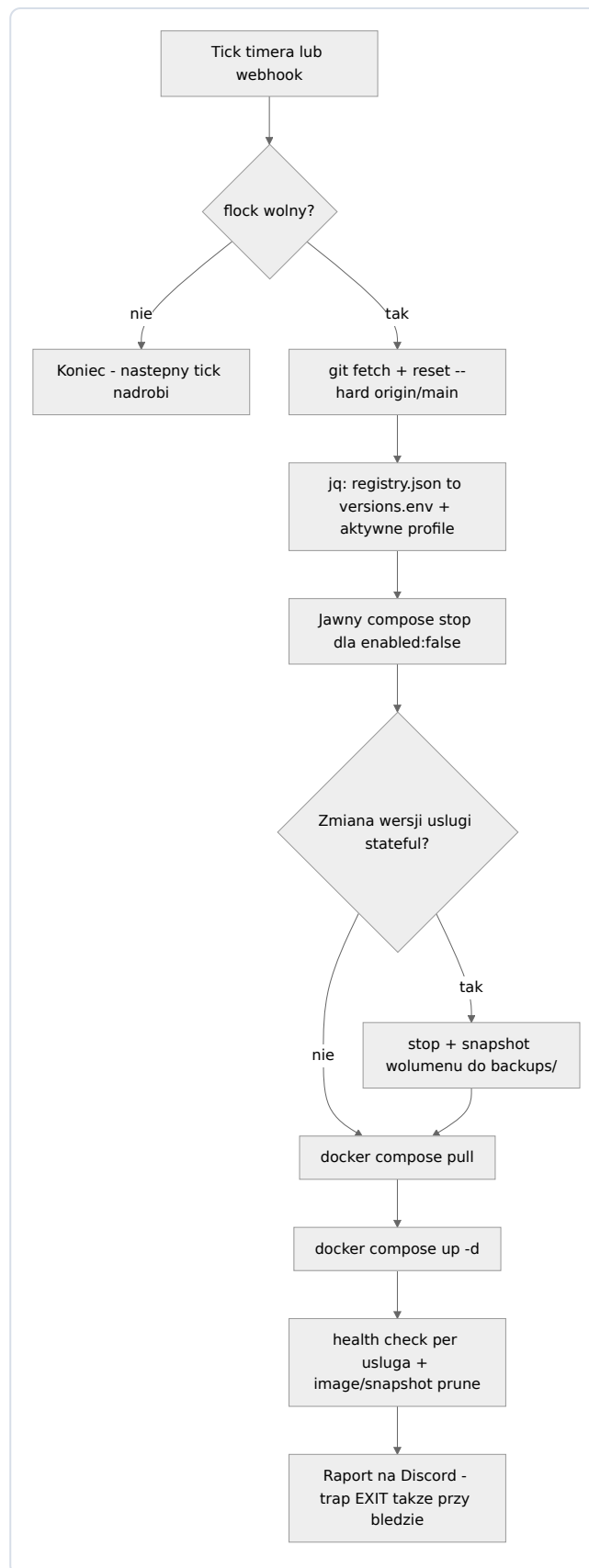


12.2. Łańcuch wydania: gdzie kończy się automat

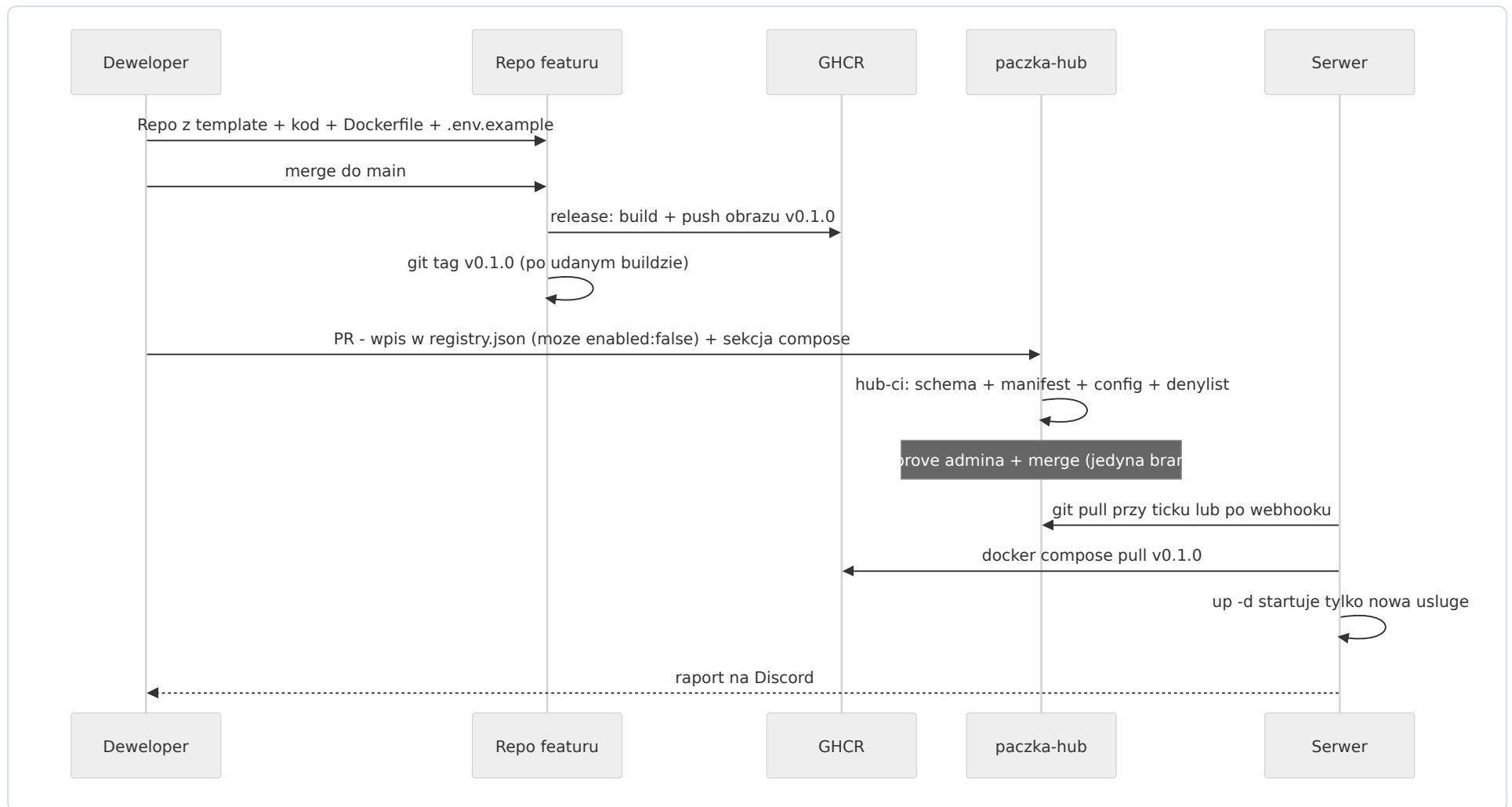
R = ręcznie, **A** = automat. Człowiek pamięta dwa ruchy: merge w featurze i bump w hubie.



12.3. Pętla reconcile

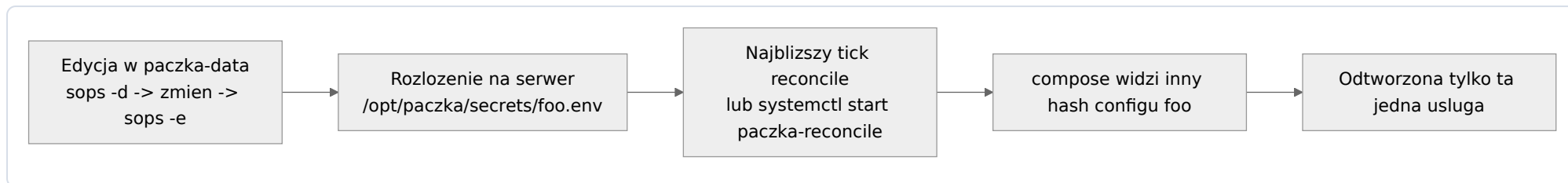


12.4. Dodanie nowego serwisu (sekwencja)

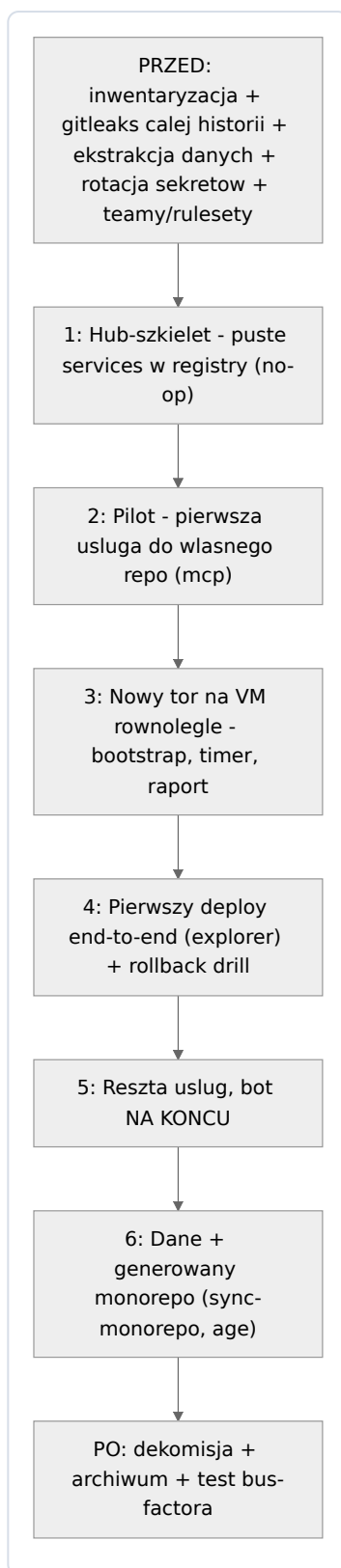


12.5. Rotacja sekretu

Kierunek zawsze `paczka-data` → serwer; brak kroku „zrestartuj ręcznie” — restart jest skutkiem różnicy wykrytej przez pętlę.



12.6. Etapy migracji



13. Plan migracji: PRZED / W TRAKCIE / PO

Strategia **duciciela**: nowy tor rośnie obok starego, przełączasz usługę po usłudze, stary monorepo archiwizujesz na końcu. Usługi działają nieprzerwanie przez całą migrację.

PRZED (przygotowanie — najważniejsze, bo nieodwracalne)

Dlaczego to krytyczne w Twoim przypadku: masz materiały i (być może) sekrety w historii jednego repo. **Upublicznienie ujawnia CAŁĄ historię** — każdy sekret z dowolnego commita wycieknie.

- **0a. Inwentaryzacja.** Wypisz: które komponenty są deployowalne (→ featur), a które to narzędzia (→ featur bez obrazu); jakie sekrety istnieją; gdzie leżą materiały. Zdecyduj **ostateczne nazwy** (nazwa huba wciąż otwarta).
- **0b. `gitleaks` na CAŁEJ historii** obecnego monorepo. Każdy hit → **rotacja** sekretu (nie wycinanie z historii — wycinanie bez rotacji to złudzenie).
- **0c. Ekstrakcja danych.** Materiały → **prywatne** `paczka-data`. To repo może zachować historię, bo zostaje prywatne.
- **0d. Strategia historii dla repo PUBLICZNYCH.** Najbezpieczniej: publiczne featury i hub startują ze **świeżym initial commitem** (albo `git filter-repo` wyciągające tylko podkatalog kodu, przeskanowane `gitleaks`). Obecny monorepo **zostaje prywatny jako archiwum** — nie upubliczniasz go.
- **0e. Organizacja i dostęp.** Utwórz teamy, rulesety, CODEOWNERS na *pustych* szkieletach repo. **Bramka:** test negatywny — ktoś spoza teamu **nie** pushnie do `main`.

W TRAKCIE

Legenda ramek: warunek wyjścia, bez którego nie idziesz dalej.

- **Etap 1 — Hub-skielet.** Postaw `paczka-hub` z **pustym** `services: {}` w registry (no-op), `reconcile.sh`, `compose` skeleton, `schema`, `hub-ci`. **Bramka:** `hub-ci` blokuje zły PR; `DRY_RUN` liczy plan (nic do zrobienia).
- **Etap 2 — Pilot: pierwsza usługa.** Kolejność `mcp` → `explorer` → `bot`. `mcp` jest dziś `enabled:false`, więc testuje łańcuch **bez dotykania produkcji**. Wyciągnij ją do własnego publicznego repo (świeża/filtrowana historia, `gitleaks` czysty); jej `release.yml` buduje+pushuje+taguje. **Bramka:** zielone CI, obraz w GHCR, smoke test.
- **Etap 3 — Nowy tor na VM równolegle.** `bootstrap.sh`, timer, `reconcile` chodzi, raport na Discord. **Stary deployment działa nietknięty.** **Bramka:** po 24h regularne przebiegi, `flock` odbija nakładki, błąd = czerwony raport.
- **Etap 4 — Pierwszy deploy end-to-end (`explorer`) + rollback drill.** **Bramka:** usługa odpowiada, `version` == registry, revert cofa obraz.
- **Etap 5 — Reszta usług, bot NA KOŃCU** (najbardziej widoczny, ciągnie webhook). **Bramka per usługa** + test: *zatrzymaj bota, zrób bump, tick i tak wdroży* (dowód, że webhook to tylko przyspieszenie, a kręgosłupem jest timer).
- **Etap 6 — Dane i monorepo.** Pełne age w `paczka-data`, `sync-monorepo`, `content-sync`. **Bramka:** nocny run składa `paczka-monorepo`, dispatch dociera do featuru.

Reguła hotfixów w trakcie: poprawka do jeszcze-nieprzetłaczanej usługi → najpierw stare repo, cherry-pick do nowego. Po przetłoczeniu → wyłącznie nowe repo; katalog w starym dostaje `PRZENIESIONE.md`.

PO (dekomisja i utrwalenie)

- **Tydzień równoległy**, potem archiwizacja starego monorepo jako **read-only** (Archive w GitHubie).
- **Test bus-factora = bramka końcowa:** osoba spoza migracji przechodzi **pełny cykl** (nowa wersja usługi + deploy + rollback) **tylko z dokumentacji**. Dopóki nie przejdzie — migracja jest *przeniesiona*, nie *skończona*.
- **Restore drill** raz na semestr (odtworzenie sekretów + VM z zera).
- **Retencja** (prune obrazów + snapshotów) działa w pętli.
- **Budżet-alert** w Oracle aktywny (patrz pułapki).

Realistyczny czas (po godzinach): PRZED + Etap 1 = jeden weekend; pilot 2-4 = drugi; reszta 2-3 tygodnie w rytmie „jedna usługa na posiedzenie”.

14. Kolejność wdrażania (co najpierw i dlaczego)

Priorytet: *najpierw tanie-a-egzystencjalne, potem ścieżka krytyczna wydania, na końcu template'y*.

Tanie, a egzystencjalne (minuty): 1. **Backup ≥ 2 kluczy adminów** w osobnych menedżerach haseł — jedyny nieredukowalny sekret DR. 2. **README huba + CODEOWNERS** — zdanie „merge do main = root na VM” i rygor na `server/` / `compose/`. 3. **Oracle → PAYG + budżet-alert ~\$1** — ucieka od odzyskiwania bezczynnych instancji (patrz pułapki).

Ścieżka krytyczna wydania (w tej kolejności): 4. `release.yml` (odwrócona kolejność) — ustanawia niezmiennik „tag $\Rightarrow$ obraz”, na którym stoi wszystko dalej; odblokowuje wydania bez GitHub App. 5. `reconcile.sh` — to jest produkcja; musi istnieć przed jakimkolwiek deploym end-to-end. 6. `hub-ci.yml` + `compose-denylist.jq` — zamienia „tag $\Rightarrow$ obraz” i bezpieczeństwo compose w maszynowo wymuszane reguły. 7. `bootstrap.sh` — czyni DR realnym, nie teoretycznym.

Higiena (~godzina): 8. `git pull` jako źródło registry (zamiast curl do raw) — znika cache CDN. 9. `prune.sh` (obrazy + snapshoty). 10. Higiena webhooka (filtr repo/ref, 200 natychmiast, async, flock).

Template'y i enablery (po udowodnieniu ścieżki na jednej usłudze): 11. `ci-node.yml` / `ci-python.yml` + zawartość `template-feature-node`. 12. `registry.schema.json`. 13. `sync-monorepo.yml` (wymaga GitHub App / PAT — jedyne takie miejsce).

15. Cykl życia: co ręcznie, co automat

Legenda: **R** = ręcznie, **A** = automat w tle.

Stale w tle - **A** timer systemd: reconcile co ~5 min (OnBootSec=2min, RandomizedDelay=30s). - **A** każdy przebieg: `git reset --hard` huba → diff registry vs docker → korekta → raport. - **A** brak zmian = przebieg nic nie robi (idempotencja).

Wydanie nowej wersji usługi - **R** PR z kodem (Conventional Commits) → **A** CI bramkuje → **R** review + merge → **A** dry-run wersji → **A** build + push `:vX.Y.Z` → **A** git tag → **R** PR do huba: bump `version` → **A** hub-ci waliduje → **R** approve + merge → **A** webhook/tick budzi reconcile → **A** compose pull + up -d (tylko ta usługa) → **A** health + prune + raport → **R** rzut oka na raport.

Nowy serwis - **R** repo z template → **R** merge do main → **A** tag v0.1.0 + obraz → **R** sekrety do `paczka-data` + serwer → **R** jeden PR do huba (wpis registry, może `enabled:false`, + sekcja compose) → **A** hub-ci → **R** approve + merge → **A** reconcile stawia usługę → **R** (opcjonalnie) drugi PR `enabled:true`.

Rotacja tokena - **R** edycja w `paczka-data` (`sops -d` → zmień → `sops -e`) → **R** rozłożenie na serwer → **A** tick: inny hash → odtworzona ta jedna usługa.

Rollback - **R** revert PR-a w hubie + approve + merge → **A** reconcile cofa obraz (stateful: snapshot sprzed bumpu chroni dane).

Wyłączenie usługi - **R** PR `enabled:false` + merge → **A** reconcile: jawny `stop`.

Człowiek pamięta dwa ruchy: **merge w featurze, bump w hubie**. Reszta to automat.

16. Pułapki (footguns)

- **Mount** `/var/run/docker.sock` — trywialny root na hoście; najwyższa reguła deny-listy compose.
- **Tag przed buildem** — literówka/bump przed dokończonym buildem → `version` wskazuje nieistniejący obraz → reconcile wywala się cicho. Rozwiązanie: odwrócona kolejność release + `docker manifest inspect` w hub-ci.
- **GITHUB_TOKEN nie wyzwala downstreamu** — tag pushnięty tym tokenem *nie* odpali kolejnego workflow. Dlatego build+tag w tym samym jobie (albo GitHub App), nie łańcuch „tag → osobny workflow”.
- **Cache CDN** `raw.githubusercontent.com` (~5 min) — dlatego czytaj registry przez `git pull`, nie `curl` do raw.
- **Samopodmiana skryptu** — `git pull` podmieniający `reconcile.sh` w trakcie jego działania. Bezpieczne TYLKO dzięki `main "$@"` w ostatniej linii.
- **enabled:false niczego nie wyłącza** — `up -d` nie zatrzyma już działającego kontenera. Konieczny jawny `compose stop`.
- **Oracle odzyskuje beczynne Always Free** (CPU/sieć/pamięć <20% przez 7 dni; logowanie SSH nie jest kryterium). Wyjście: **Pay As You Go** (Always Free dalej darmowe,

odzyskiwanie znika). Dociążanie CPU cronem = zakazane obejście.

- **PAYG nie da twardego zera.** Budżety OCI są *tylko* alertujące (ewaluacja co 24h). Twardy limit istnieje jako DIY (budżet → Events → Function tworząca zerową quote), ale blokuje tylko *tworzenie nowych* zasobów, nie zatrzymuje działających, i ma lag. **Realna ochrona:** provisionuj wyłącznie kształty Always-Free-eligible (koszt = \$0 z definicji) + budżet-alert ~\$1, by *pomyłkę* złapać w dobie. 10 TB egressu/mies. to duży zapas.
- **Health ≠ „up”.** Kontener „up” ≠ zdrowy. Konieczny realny `healthcheck:` + odczyt `State.Health.Status` per usługa w reconcile.
- **Dysk** — dwa źródła „no space left”: obrazy i snapshoty. Obie retencje w pętli.
- **Rollback zakłada bezstanowość** — dla usług stateful: named volume nietykany przez reconcile + kontrakt „kod vN-1 działa na schemacie vN” (expand-contract, forward-only) + snapshot przed bumpem. Zmiana łamiąca to *restore z backupu*, nie rollback.

17. Lista podsumowująca (TL;DR)

Zapamiętaj przede wszystkim: - `registry.json` opisuje stan; serwer sam go dogania (pull, nie push). - Wdrożenie = bump `version` + approve PR w hubie. **Jedna brama.** - Publiczne wszystko poza `paczka-data` — bo tylko na publicznych działają darmowe rulesety, minuty Actions i anonimowy pull GHCR. - **Merge do main huba = root na VM** — zapisz w README, chroń CODEOWNERS-em. - Tag powstaje **po** buildzie → „tag istnieje ⇒ obraz istnieje”. - Sekrety: age/SOPS do wielu odbiorców; **przetrwanie ≥1 klucza admina = pełne DR**; klucz serwera jest odtwarzalny. - `up -d` bez `down`; `enabled:false` = jawny `stop`; health per usługa; prune obrazów i snapshotów w pętli.

Zanim upublicznisz cokolwiek: - `gitLeaks` na całej historii → rotacja każdego znalezionejgo sekretu. - Materiały → prywatne `paczka-data`. - Repo publiczne startują ze świeżą/filtrowaną historią; obecny monorepo → prywatne archiwum. - Teamy + rulesety + CODEOWNERS; test negatywny (spoza teamu brak push do main).

Kolejność: klucze adminów + README/CODEOWNERS + PAYG → `release.yml` → `reconcile.sh` → `hub-ci` + deny-lista → `bootstrap.sh` → higiena → template'y → `sync-monorepo`.

Bramka końcowa migracji: osoba spoza migracji robi pełny cykl **tylko z tej dokumentacji**. Dopóki nie przejdzie — migracja jest przeniesiona, nie skończona.